Лабораторна робота №1

Тема:

CI/CD з GitHub Actions: автоматизація збірки та тестування

Мета:

Ознайомитися з сервісом GitHub Actions, його архітектурою та можливостями. Навчитися створювати базовий pipeline для автоматичної збірки проєкту та запуску тестів при кожному оновленні репозиторію.

Завдання:

1. Вивчити базову термінологію та архітектуру GitHub Actions (workflow, job, step, runner, event).
2. Створити публічний або приватний репозиторій на GitHub для власного проєкту.
3. Додати каталог `.github/workflows/` та створити у ньому файл `ci.yml`.
4. Реалізувати workflow, який:
    - запускається при події `push` до гілки `main`;
    - виконує клонування репозиторію (`actions/checkout`);
    - встановлює залежності (npm або pip);
    - запускає тести.
5. Перевірити результат виконання workflow через GitHub UI.

Короткі теоретичні відомості:

CI/CD (*Continuous Integration / Continuous Delivery or Deployment*) — це набір практик, спрямованих на автоматизацію процесу збірки, тестування та доставки програмного забезпечення.

GitHub Actions — це вбудований інструмент у GitHub для створення CI/CD-процесів без використання сторонніх сервісів, таких як Jenkins чи Travis CI.

Основні поняття GitHub Actions:

- **Workflow** — це автоматизований процес, який виконується внаслідок певної події (наприклад, push до репозиторію).
- **Event** — тригер, який запускає workflow (наприклад, `push`, `pull_request`, `schedule`).
- **Job** — логічний блок роботи, який виконується на окремій віртуальній машині (runner).
- **Step** — окрема команда в межах job, яка виконується послідовно.
- **Runner** — середовище (віртуальна машина), де запускається job. GitHub пропонує хмарні (hosted) або самостійні (self-hosted) runner'и.

Файл workflow пишеться у форматі **YAML** і розміщується у каталозі:

```
.github/workflows/
```

Приклад базового workflow для Node.js:

```
name: CI

on:
  push:
    branches: [main]

jobs:
  build:
    runs-on: ubuntu-latest

    steps:
      - uses: actions/checkout@v3
      - name: Install dependencies
        run: npm install
      - name: Run tests
        run: npm test
```

Хід виконання роботи:

Крок 1. Підготовка репозиторію

1. Створіть новий репозиторій на GitHub (можна публічний або приватний).
2. Ініціалізуйте його з файлом `README.md`.

Крок 2. Клонування репозиторію

1. Склонуйте репозиторій на свій локальний комп'ютер:

```
git clone https://github.com/<ваш_логін>/<назва_репозиторію>.git
cd <назва_репозиторію>
```

Крок 3. Створення структури workflow

1. Створіть каталоги:

```
mkdir -p .github/workflows
```

2. Створіть файл `ci.yml` у створеній папці:

```
touch .github/workflows/ci.yml
```

Крок 4. Написання workflow

1. Відкрийте файл `ci.yml` у редакторі коду та вставте конфігурацію (наприклад, для Python-проєкту):

```
name: Python CI

on:
  push:
    branches: [main]

jobs:
  build:
    runs-on: ubuntu-latest

    steps:
      - uses: actions/checkout@v3

      - name: Set up Python
        uses: actions/setup-python@v4
        with:
          python-version: '3.10'

      - name: Install dependencies
        run: |
          python -m pip install --upgrade pip
          pip install -r requirements.txt

      - name: Run tests
        run: |
          pytest
```

Крок 5. Додавання тестів (за потреби)

1. Додайте в корінь проєкту файл `requirements.txt` (наприклад, для Python):

```
pytest
```

2. Додайте приклад тесту у `test_example.py`:

```
def test_sum():
    assert 2 + 2 == 4
```

Крок 6. Коміт та пуш

1. Закомітіть зміни та відправте у репозиторій:

```
git add .
git commit -m "Add CI workflow"
git push origin main
```

Крок 7. Перевірка результату

1. Перейдіть у вкладку **Actions** на сторінці репозиторію GitHub.

2. Перевірте, що workflow виконався без помилок.
3. У разі помилки — відкрийте лог, знайдіть step з помилкою і виправте його.

Контрольні запитання:

1. Яка структура workflow-файлу у GitHub Actions?
2. Яка роль runner'a?
3. У чому переваги GitHub Actions перед іншими CI/CD-інструментами?
4. Як можна організувати паралельне виконання job'ів?